A Small-Circle Social Media Platform

Qiwen Lin · Irys Zhang · Weijie Zhu · Zhengyang Li

University of Toronto — CSC309 Winter 2026

The Problem

Mainstream social media encourages quantity over quality

- Platforms like Instagram, Twitter, Facebook prioritize **large follower counts** and engagement metrics
- Users experience **shallow interactions**, social comparison, and content overload
- Research shows people maintain meaningful relationships with only **5–15 close friends** (Dunbar's Number)

Existing solutions fall short

Platform	Limitation
BeReal	Still open follower model, no cap
IG Close Friends	An afterthought, not core design
WhatsApp Groups	No social feed, profiles, discovery

> **None of these structurally enforce small, meaningful networks.**

Our Solution: InnerCircle

A social platform that limits your circle to ~20–30 friends

Core Concept

- **Enforced friend cap** — must remove a friend to add a new one at the limit
- Every interaction happens within a **small, trusted group**
- Social media that feels **personal, not performative**

Target Users

- Young adults and students
- Want to share with close friends through **photo sharing** and **real-time messaging**
- Without the noise of large-scale social networks

Key Differentiators

1. Finite Social Graph — Friend cap is a *structural constraint*, not just a config flag. Enforced at database transaction level.

2. Friend-Based Access Control — Every post, message, and notification is gated by friendship status.

3. Real-Time First — Chat, notifications, online/offline status all delivered via WebSockets.

Technical Architecture

Next.js Full-Stack Application (Monorepo)

Frontend

Next.js 16 React 19 TypeScript

Tailwind CSS shadcn/ui

Redux Toolkit Socket.io Client

Backend

Next.js API Routes REST API

PostgreSQL Prisma ORM

Better Auth Socket.io Server

AWS S3 / DO Spaces Sharp

Architecture Overview

```
Browser (React + Redux + Socket.io Client)
    │
    ▼
Custom HTTP Server (server.ts)
    ├── Next.js Handler (API Routes + Pages)
    │       │
    │       ▼
    │   Prisma ORM ──→ PostgreSQL
    │       │
    │       ▼
    │   AWS S3 (Image Storage)
    │
    └── Socket.io Server
            │
            ▼
        Real-time Events
        (chat, notifications, presence)
```

> **Key Decision:** Custom HTTP server wraps Next.js to support persistent WebSocket connections.

Database Design

10 models — relational schema managed with Prisma ORM + PostgreSQL

Core Models

Model	Purpose	Key Fields
`User`	User profiles	name, email, avatar, bio, **friendCapLimit**
`Friendship`	Friend graph	requesterId, addresseeId, status (enum)
`Post`	User posts	textContent, lat, lng, authorId
`PostImage`	Post media	imageUrl, thumbnailUrl, orderIndex
`Comment`	Comments	content, postId, authorId
`Like`	Likes	postId, userId (unique constraint)

Real-Time Models

Model	Purpose	Key Design
`Conversation`	Chat sessions	Avoids N+1 query problem
`Message`	Chat messages	readAt for read receipts
`Notification`	Alerts	6 types, read/unread flag

Key Design Decisions

- `Friendship` : self-referencing M:N with **unique(requesterId, addresseeId)**
- `Like` : **unique(postId, userId)** prevents double-likes
- Strategic **indexes** on foreign keys, timestamps, and composite fields
- `Conversation` added to avoid flat message query inefficiency

Core: Authentication & Authorization

Better Auth with session-based authentication + centralized access control

Authentication Flow

Register / Login → Better Auth validates → Session created → Cookie set

Implementation

- Session-based auth (7-day expiry, 1-day refresh)
- Password minimum 8 characters
- Middleware: unauthenticated → `/login`
- Middleware: authenticated on auth pages → `/feed`

```
POST /api/auth/sign-up    → Register
POST /api/auth/sign-in    → Login
GET  /api/auth/session     → Current session
POST /api/auth/sign-out    → Logout
```

Centralized Auth Utilities

```
// src/lib/auth-utils.ts

// For Server Components (redirects)
async function requireSession()

// For API routes (throws 401)
async function requireSessionForApi()

// General session check
async function getSession()
```

> **Security Pattern:** Every API route starts with `requireSessionForApi()`. One file to audit — `src/lib/auth-utils.ts`.

Route Protection

- Middleware intercepts all non-public, non-API routes
- Session cookie checked via `getSessionCookie()`
- Public paths: `/`, `/login`, `/register`

Core: Friend System with Cap Enforcement

The defining feature — structural friend limit enforced at database level

Friend Request Flow

Search user → Check cap → Send request →

Addressee checks cap → Accept / Decline

Cap Enforcement

- Default cap: **25 friends** per user
- Cap checked on **both** sending AND accepting
- At cap → must remove a friend before adding new one
- UI shows **progress bar**: "12 / 25 friends"

Race Condition Protection

```
// Atomic transaction prevents concurrent
// accepts from exceeding the cap
await prisma.$transaction(async (tx) => {
  const count = await tx.friendship.count({ ... })
  if (count ≥ user.friendCapLimit) throw Error
  await tx.friendship.update({status: "ACCEPTED"})
})
```

API Endpoints

Method	Endpoint	Action
GET	`/api/friends`	List friends
POST	`/api/friends/request`	Send request
POST	`/api/friends/respond`	Accept/Decline
POST	`/api/friends/remove`	Unfriend
GET	`/api/friends/pending`	Pending list
GET	`/api/users/search`	Search users

Shared Utilities (`src/lib/friends.ts`)

- `getFriendIds()` — with TTL cache
- `areFriends()` — friendship check
- `getFriendCount()` — count for cap
- `invalidateFriendCache()` — cache busting

> **Access Control:** Every API route that involves user content checks friendship status via `areFriends()` .

Core: Posts, Feed & Social Interactions

Friend-based feed with like/comment system and cloud storage

Feed Logic

- Query posts from **self + all friends only**
- **Cursor-based pagination** (20 posts/page)
- Each post: author info, images, like/comment counts, `isLiked` flag
- Friend-based access control on every endpoint

API Endpoints

Method	Endpoint	Action
GET	`/api/posts`	Feed (paginated)
POST	`/api/posts`	Create post
POST	`/api/posts/[id]/like`	Toggle like
GET	`/api/posts/[id]/comments`	List comments
POST	`/api/posts/[id]/comments`	Add comment

Post Creation Flow

Write text → Upload images → Process & store → Create post record

Key Implementation Details

- Posts support **multiple images** with order indexing
- Like uses **unique(postId, userId)** constraint — toggle behavior
- Like/comment triggers **notification** to post author
- Notification also emitted via **Socket.io** for real-time delivery

> **Cloud Storage:** Post images stored in DigitalOcean Spaces (S3-compatible). Both full-size and thumbnail URLs saved in `PostImage` model.

Advanced Feature 1: Real-Time Chat

Socket.io — One-on-one messaging with typing indicators and read receipts

Architecture

```
// server.ts (project root)
const httpServer = createServer(handler)
initSocketServer(httpServer)  // Attach Socket.io
httpServer.listen(3000)

// userSockets: Map<userId, Set<socketId>>
// Supports multiple tabs per user
```

Socket Events

Event	Direction	Purpose
`chat:message`	↔	Send/receive message
`chat:typing`	→	Typing indicator
`chat:read`	→	Mark as read
`user:online`	←	User connected
`user:offline`	←	User disconnected

Message Persistence

- Messages saved to PostgreSQL via Prisma **before** being emitted
- `Conversation` model for efficient listing and unread counts
- `readAt` timestamp for read receipt tracking
- Message history loaded from API, new messages via Socket

Client Integration

- `SocketProvider` connects on login, disconnects on logout
- Incoming events dispatched to **Redux** slices
- Instant UI updates across all components

> **Design Decision:** Custom HTTP server chosen from Day 1 (not a fallback) — Next.js API Routes cannot handle WebSocket upgrades. This eliminates the integration risk entirely.

Advanced Feature 2: Image Processing Pipeline

Server-side compression, WebP conversion, and thumbnail generation with Sharp

Pipeline Flow

Client Upload (≤10MB) → Validate type & size → Sharp: resize + WebP → Sharp: thumbnail → Upload to S3 → Return URLs

Post Image Processing

```
// Full size: max 1920px, WebP @ 80%
sharp(input)
  .resize(1920, 1920, { fit: "inside" })
  .webp({ quality: 80 })
  .toBuffer()

// Thumbnail: 400x400 crop, WebP @ 60%
sharp(input)
  .resize(400, 400, { fit: "cover" })
  .webp({ quality: 60 })
  .toBuffer()
```

Avatar Processing

```
// 256x256 crop, WebP @ 80%
sharp(input)
  .resize(256, 256, { fit: "cover" })
  .webp({ quality: 80 })
  .toBuffer()
```

Storage

- AWS SDK v3 for **S3-compatible** storage (DigitalOcean Spaces)
- Organized folders: `posts/` , `posts/thumbnails/` , `avatars/`
- UUID-based filenames prevent collisions
- Public-read ACL for direct URL access

> **Design Decision:** Compress *before* upload to S3 (not after). Client-side 10MB limit keeps Sharp's work manageable. No background job queue needed.

Advanced Features 3 & 4

AI Content Moderation (External API)

Non-blocking, fail-open design

Upload succeeds → Async moderation → Flag if unsafe

- Image uploads succeed **immediately**
- Moderation runs in the **background**
- 10-second timeout prevents hanging
- API down → **fail open** (allow image)
- Flagged images logged for review

```
// Fire-and-forget pattern
moderateImage(url).then((result) => {
  if (!result.safe)
    console.warn("Flagged:", result.categories)
})
```

> Satisfies **External API Integration** requirement.

Redux Toolkit State Management

Global state for real-time data

Slice	Purpose
`feedSlice`	Posts, pagination, optimistic likes
`chatSlice`	Conversations, messages, typing
`notificationSlice`	Alerts, unread count
`friendSlice`	Friends, requests, online status

Why Redux over React Context?

- Multiple real-time data sources (WebSocket + API + optimistic updates)
- `createAsyncThunk` for API calls with loading/error
- Socket.io events dispatch directly → instant UI updates
- No race conditions or stale data

> Satisfies **Advanced State Management** requirement.

Notification System & Frontend Design

Real-Time Notifications

6 notification types delivered instantly

Type	Trigger
`FRIEND_REQUEST`	Someone sends a friend request
`FRIEND_ACCEPT`	Friend request accepted
`NEW_POST`	Friend creates a new post
`LIKE`	Someone likes your post
`COMMENT`	Someone comments on your post
`MESSAGE`	New direct message received

Delivery

- Created in DB via API routes
- Emitted via Socket.io to online users
- Redux `notificationSlice` updates unread count
- Badge indicator on navigation

Frontend UI Design

Responsive layout — desktop + mobile

- **Desktop:** Left sidebar navigation
- **Mobile:** Bottom tab bar (44px touch targets)

Key Pages

- **Feed** — Infinite scroll, post cards with images, like/comment
- **Friends** — 3 tabs (My Friends, Requests, Search) + cap progress bar
- **Chat** — Two-panel layout (conversations + active chat)
- **Notifications** — Real-time list with unread badge
- **Profile** — Avatar, bio, friend count, post history, edit dialog

Component Library

shadcn/ui Tailwind CSS lucide-react icons

Implementation Status & Completion Plan

Current Status

Module	Status
Database & Prisma Schema (10 models)	✅ Complete
Authentication (Better Auth)	✅ Complete
Friend System API (cap enforcement)	✅ Complete
Posts & Feed API (CRUD + access control)	✅ Complete
Real-Time Chat (Socket.io + persistence)	✅ Complete
Notification System (6 types + real-time)	✅ Complete
Image Pipeline (Sharp + S3)	✅ Complete
AI Content Moderation	✅ Complete
Frontend UI Pages	🔧 In Progress
Redux Integration	🔧 In Progress

Completion Plan (Mar 19 → Apr 3)

Week of Mar 19–25

- Complete all frontend pages (Feed, Friends, Chat, Notifications, Profile)
- Wire Redux slices to Socket.io events
- Integrate image upload into post creation and profile edit

Week of Mar 26 – Apr 1

- Responsive design polish (mobile optimization)
- Error handling & edge cases
- Performance optimization (lazy loading, pagination)

Apr 1–3

- End-to-end testing of all user flows
- Documentation (README, ai-session.md)
- Video demo recording

> **All backend APIs and advanced features are functional.** Remaining work is frontend UI and integration.

Thank You

Social media that feels personal, not performative.

Qiwen Lin · Irys Zhang · Weijie Zhu · Zhengyang Li

Questions?